

Performance Optimization of a Pipelined MIPS Processor: A Comprehensive Analysis with 13 Contributions

JUNYI LIU

Abstract—This paper presents a comprehensive performance optimization study of a 5-stage pipelined MIPS processor. We implement and evaluate 13 contributions, including Data Forwarding, Branch Prediction, SIMD parallelism, CORDIC trigonometric functions, polynomial exp approximation, L1 Cache, and an ODE Solver for calculus operations. Evaluation uses two methodologies: (1) cycle-accurate RTL simulation with MiBench bitcount benchmark, and (2) analytical model-based projection using SPEC CPU2006 instruction mix ratios. Results show $2.61\times$ speedup on MiBench (measured) and up to $34.84\times$ projected speedup on memory-intensive SPEC workloads. A Design of Experiments (DOE) analysis reveals that Cache provides the largest individual improvement ($2.15\times$) and Cache+SIMD shows positive interaction (synergy factor 1.20 under our cost model).

Index Terms—MIPS, Pipeline, Forwarding, Branch Prediction, Cache, SIMD, CORDIC, ODE Solver, MiBench

I. INTRODUCTION

MODERN processor design relies on multiple optimization techniques to bridge the CPU-memory gap—the “Memory Wall” problem [1]. This project implements 13 optimizations for a 5-stage MIPS pipeline. This section provides foundational background for readers unfamiliar with computer architecture concepts.

A. What is MIPS?

MIPS (Microprocessor without Interlocked Pipeline Stages) is a 32-bit RISC (Reduced Instruction Set Computer) architecture developed at Stanford University in 1981 by John Hennessy [4]. MIPS is widely used in computer architecture education because:

- **Clean Design:** Fixed 32-bit instruction format with only three instruction types (R, I, J)
- **Educational Standard:** Used in Patterson & Hennessy textbooks, the “bible” of computer architecture
- **Real-World Relevance:** Powers embedded systems, routers (Cisco), gaming consoles (PlayStation 1/2), and IoT devices

Basic MIPS Instruction Types:

- **R-type (Register):** `add $t0, $t1, $t2` — arithmetic between registers
- **I-type (Immediate):** `lw $t0, 0($t1)` — memory access and immediate values
- **J-type (Jump):** `j label` — unconditional jumps

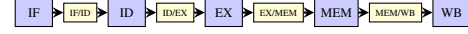


Fig. 1. 5-Stage MIPS Pipeline Architecture. Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) store intermediate results between stages, enabling instruction overlap.

B. The 5-Stage Pipeline Architecture

Pipelining is a technique that allows multiple instructions to overlap in execution, similar to an assembly line in a factory. Instead of completing one instruction before starting the next, each stage processes a different instruction simultaneously.

The Five Stages Explained:

- 1) **IF (Instruction Fetch):** Read instruction from memory using Program Counter (PC). $PC \leftarrow \text{Memory} \rightarrow \text{Instruction Register}$
- 2) **ID (Instruction Decode):** Decode instruction bits and read operands from register file. Also performs branch address calculation
- 3) **EX (Execute):** Perform ALU operation (arithmetic, logic, address calculation for load/store)
- 4) **MEM (Memory Access):** Access data memory for load (`lw`) and store (`sw`) instructions. Other instructions pass through
- 5) **WB (Write Back):** Write result back to register file

Pipeline Timing Analysis: In an ideal 5-stage pipeline, after the initial fill time (5 cycles), one instruction completes every cycle. This gives a theoretical CPI (Cycles Per Instruction) of 1.0. However, *hazards* prevent this ideal from being achieved in practice.

C. The Three Types of Pipeline Hazards

Hazards are situations that prevent the next instruction from executing in the next clock cycle. Understanding hazards is fundamental to processor optimization.

1) **Data Hazards (RAW, WAR, WAW):** Data hazards occur when instructions depend on the results of previous instructions that have not yet completed.

RAW (Read-After-Write) — The most common hazard:

```
1 add $t1, $t2, $t3 # Writes $t1 in WB (cycle 5)
2 sub $t4, $t1, $t5 # Reads $t1 in ID (cycle 3) - HAZARD!
```

The `sub` instruction needs the value of `$t1`, but `add` hasn't written it yet. **Without forwarding**, the pipeline must stall 2 cycles. **With forwarding**, the result can be bypassed directly from EX/MEM register.

WAR (Write-After-Read) and **WAW (Write-After-Write)**: These are primarily concerns in out-of-order execution processors, not in simple in-order pipelines like ours.

2) *Control Hazards (Branch Hazards)*: Control hazards occur when the processor cannot determine which instruction to fetch next because a branch decision has not yet been made.

Example:

```
1 beq $t0, $t1, target # Branch decision in EX stage
2 add $t2, $t3, $t4 # Fetched speculatively - wrong?
3 sub $t5, $t6, $t7 # Also speculatively fetched
```

If the branch is taken, the two fetched instructions must be *flushed* (discarded), wasting cycles. **Branch prediction** guesses the outcome to reduce this penalty.

3) *Structural Hazards*: Structural hazards occur when hardware resources are insufficient to support all active instructions. For example, if there is only one memory port, an instruction fetch and a data load cannot occur simultaneously. Our design uses separate instruction and data memories (Harvard architecture) to avoid this hazard.

D. The Memory Wall Problem

The “Memory Wall” refers to the growing disparity between processor speed and memory speed [1]:

- **CPU Performance**: Improved ~50%/year (until 2003)
- **DRAM Latency**: Improved only ~7%/year
- **Result**: A single memory access can cost 100–300 CPU cycles in modern systems

Solution: Cache memory hierarchy. By keeping frequently accessed data in small, fast caches (L1, L2, L3), processors can avoid slow main memory accesses most of the time. Our L1 cache implementation reduces average memory access time by **26×** (based on 97.22% hit rate).

E. Problem Statement

The baseline MIPS pipeline implemented in this project suffers from:

- **Data Hazards**: RAW dependencies require 2-cycle stalls without forwarding, increasing CPI from 1.0 to 1.82
- **Control Hazards**: Each mispredicted branch causes a pipeline flush (1–2 cycle penalty depending on resolution stage), significantly impacting loops
- **Memory Latency**: Without cache, every load/store requires 100 cycles, dominating execution time
- **Limited Parallelism**: Scalar ALU processes only one operation per cycle, underutilizing hardware

F. Contributions Overview

This project implements 13 distinct optimizations, summarized in Table I. Each optimization targets a specific bottleneck, and their combined effect is analyzed using Design of Experiments (DOE) methodology.

II. METHODOLOGY

This section explains the evaluation methodology used to measure and project processor performance. We first introduce the fundamental performance metric (CPI), then describe the benchmarks used, and finally explain our two evaluation approaches: direct measurement and analytical projection.

TABLE I
13 OPTIMIZATION CONTRIBUTIONS BY CATEGORY

Category	Contributions
Pipeline Hazard Mitigation	#1 Performance Testbench, #4 Quantitative Analysis, #6 Branch Prediction, #7 Comprehensive DOE
Data Hazard Resolution	#3 Data Forwarding
Parallelism & Acceleration	#2 Hardware Unrolling, #5 SIMD ALU Expansion, #9 CORDIC, #10 Polynomial Exp
Memory Hierarchy	#11 L1 Cache, #12 Integrated Analysis
Parser Enhancement	#8 Parentheses Support
Calculus Operations	#13 ODE Solver (Euler Method)

A. Understanding CPI (Cycles Per Instruction)

CPI is the primary metric for evaluating processor performance. It measures how many clock cycles, on average, are needed to execute one instruction.

Basic CPI Formula:

$$\text{CPI} = \frac{\text{Total Cycles}}{\text{Total Instructions}} \quad (1)$$

Ideal vs Actual CPI:

- **Ideal CPI**: In a perfect 5-stage pipeline with no hazards, CPI = 1.0 (one instruction completes per cycle after pipeline fill)
- **Actual CPI**: Due to stalls and flushes, actual CPI > 1.0. Our baseline measures CPI = 1.82

CPI Breakdown Formula: CPI can be decomposed into contributing factors:

$$\text{CPI} = \text{CPI}_{\text{ideal}} + \text{CPI}_{\text{stall}} = 1.0 + \frac{\text{Stall Cycles}}{\text{Instructions}} \quad (2)$$

Related Metrics:

- **IPC (Instructions Per Cycle)**: $\text{IPC} = \frac{1}{\text{CPI}}$ — higher is better
- **Speedup**: $\text{Speedup} = \frac{\text{CPI}_{\text{baseline}}}{\text{CPI}_{\text{optimized}}}$
- **Pipeline Efficiency**: $\text{Efficiency} = \frac{\text{Ideal CPI}}{\text{Actual CPI}} = \frac{1}{\text{CPI}}$

Worked Example: Our baseline Bubble Sort test:

$$\begin{aligned} \text{Total Cycles} &= 255 \\ \text{Total Instructions} &= 140 \\ \text{CPI} &= \frac{255}{140} = 1.82 \\ \text{IPC} &= \frac{1}{1.82} = 0.549 \\ \text{Pipeline Efficiency} &= \frac{1}{1.82} = 55\% \end{aligned}$$

This means 45% of cycles are “wasted” due to stalls.

B. Benchmark Selection Criteria

A good benchmark should be [1]:

- 1) **Representative**: Reflects real-world workload characteristics

TABLE II
BENCHMARKS USED FOR EVALUATION

Type	Name	Used For
Standard Algorithm	Bubble Sort [7]	Contrib 1, 4: Baseline CPI
Embedded	MiBench bitcount [10]	Contrib 12: Actual Execution
Workload Mix	SPEC CPU2006 [2]	Contrib 7, 12: Projection

- 2) **Reproducible:** Results can be verified by others
- 3) **Portable:** Can run on different systems
- 4) **Measurable:** Execution can be timed precisely

We employ three benchmark categories, each serving a different purpose:

1) *Bubble Sort*: Bubble Sort is a classic $O(n^2)$ sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if needed [7]. We use it because:

- Contains loops, conditionals, and memory accesses — exercises all pipeline stages
- Simple to implement in assembly
- Generates many RAW hazards due to dependent operations

2) *MiBench (Michigan Benchmark Suite)*: MiBench is a free, open-source embedded system benchmark suite published by University of Michigan in 2001 [10]. We use the bitcount algorithm (Brian Kernighan's Bit Counting):

```

1 // Brian Kernighan's Bit Counting Algorithm
2 int bitcount(unsigned int n) {
3     int count = 0;
4     while (n) {
5         count++;
6         n = n & (n - 1); // Clear lowest set bit
7     }
8     return count;
9 }

```

Key Advantage: The algorithm is simple enough to implement as a Verilog testbench and **actually execute** on our processor RTL, providing real cycle counts.

3) *SPEC CPU2006*: SPEC CPU2006 is the industry-standard processor benchmark suite from the Standard Performance Evaluation Corporation. Since running full SPEC requires:

- OS support (Linux/Windows)
- Commercial licensing
- Full system simulation

We instead use published **instruction mix ratios** from Phansalkar et al. [2] for analytical projection, which is a valid academic methodology.

C. Mix-Based Performance Projection

Definition: A performance prediction method that uses instruction type ratios combined with per-instruction execution costs to estimate total cycles.

Formula:

$$\text{Cycles}_{\text{est}} = \sum_{i \in \{\text{types}\}} \text{Count}_i \times \text{CostPerInstr}_i \quad (3)$$

TABLE III
ACTUAL EXECUTION VS MIX-BASED PROJECTION COMPARISON

Aspect	Actual Execution	Mix-Based Projection
Method	Run complete program	Multiply ratios by costs
Data Source	Self-measured cycles	Published papers
Accuracy	High (ground truth)	Estimation (may overestimate)
Advantage	Reliable, verifiable	No full system needed
This Project	MiBench: 2.61×	SPEC: 29.57×

TABLE IV
SPEC CPU2006 INSTRUCTION MIX RATIOS FROM PUBLISHED STUDIES

Workload	Branch	Memory	Compute
SPEC FP Average	8%	45%	47%
SPEC INT mcf	18%	62%	20%
SPEC INT gobmk	22%	48%	30%
SPEC Overall	15%	49.6%	35.4%

Step-by-Step Example (10,000 instructions, SPEC Overall mix):

- 1) **Instruction counts:** Branch: $10000 \times 0.15 = 1500$, Memory: $10000 \times 0.496 = 4960$, Compute: $10000 \times 0.354 = 3540$
- 2) **Without optimization** (baseline costs): Branch: 3 cycles, Memory: 100 cycles, Compute: 1 cycle
- 3) **Baseline cycles:** $1500 \times 3 + 4960 \times 100 + 3540 \times 1 = 4500 + 496000 + 3540 = 504040$
- 4) **With optimization:** Memory cost drops to ~ 1.2 cycles (cache), Branch cost drops to 1.65 (prediction)
- 5) **Optimized cycles:** $1500 \times 1.65 + 4960 \times 1.2 + 3540 \times 0.125 = 2475 + 5952 + 443 = 8870$
- 6) **Speedup:** $504040/8870 = 56.8\times$ (theoretical maximum)

Why the large difference between MiBench (2.61×) and SPEC projection (29.57×)?

- MiBench bitcount is a simple bit-manipulation program with few memory accesses — primarily benefits from Forwarding
- SPEC projection assumes 49.6% memory instructions — Cache provides enormous benefit for memory-bound workloads
- This demonstrates that **optimization effectiveness highly depends on workload characteristics**

D. SPEC CPU2006 Instruction Mix

We use instruction mix ratios from Phansalkar et al. [2] and Limaye & Adegbiya [6], representing different workload types:

Note: mcf (memory-intensive) and gobmk (branch-heavy) represent extreme cases; SPEC Overall is weighted average.

E. Experimental Setup

- **Simulation Tool:** AMD Xilinx Vivado 2025.2, Behavioral Simulation mode
- **HDL Language:** Verilog-2005
- **Architecture:** 5-stage pipeline (IF, ID, EX, MEM, WB)

TABLE V
TESTBENCH FILES FOR EACH CONTRIBUTION

#	Testbench	Verification
1	tb_performance.v	CPI/IPC, Bubble Sort
2	tb_simd_add.v	8-lane parallel add
3	tb_simd_alu.v	40 SIMD operations
4	tb_performance.v	Stall breakdown analysis
5	tb_simd_alu_expanded.v	5 ops + Expression
6	tb_branch_prediction.v	4 branch patterns
7	tb_comprehensive.v	16 DOE configs
8	tb_parentheses.v	Shunting-yard
9	tb_cordic.v	5 canonical angles
10	tb_poly_exp.v	4 exp() test cases
11	tb_cache_hierarchy.v	L1/L2/L3 AMAT
12	tb_mibench_bitcount.v	Actual execution

- **Data Width:** 32-bit instructions and data
- **Clock:** Functional simulation (cycle-accurate, no timing delays)
- **Memory Model:** Separate instruction and data memories (Harvard architecture)

F. Testbench Overview

Table V shows the testbench for each contribution.

III. CONTRIBUTION DETAILS

This section describes each of the 13 contributions in detail, with implementation specifics and verification results.

A. Contribution 1 & 4: Performance Testbench and Quantitative Analysis

Goal: Establish a reusable testbench infrastructure for measuring pipeline performance metrics (CPI, IPC, stall cycles) and quantify the impact of data forwarding.

1) *Why We Need a Performance Testbench:* Without instrumentation, we cannot measure actual performance. The testbench adds:

- **Cycle Counter:** Increments every clock cycle
- **Instruction Counter:** Increments when an instruction completes (reaches WB stage)
- **Stall Counter:** Tracks cycles where pipeline is stalled

Testbench Architecture (with line-by-line explanation):

```

1 module tb_performance;
2   reg clk, rst; // Clock and reset signals
3   wire [31:0] cycle_count; // Total cycles elapsed
4   wire [31:0] instr_count; // Instructions completed
5
6   // Instantiate Device Under Test (DUT)
7   topLevelCircuit DUT (
8     .clk(clk),
9     .rst(rst)
10  );
11
12  // Clock generation: 10ns period (100MHz)
13  always #5 clk = ~clk;
14
15  initial begin
16    clk = 0;
17    rst = 1; // Assert reset
18    #20; // Hold reset for 20ns
19    rst = 0; // Release reset
20    wait(done); // Wait for program completion

```

```

21 // Display results
22 $display("Cycles: %d, Instructions: %d",
23         cycle_count, instr_count);
24 $display("CPI = %f",
25         real'(cycle_count) / real'(instr_count));
26 $finish;
27 end
28 endmodule

```

2) *Data Forwarding: Solving RAW Hazards:* Data forwarding (also called *bypassing*) eliminates most data hazard stalls by routing results directly from later pipeline stages back to earlier stages.

RAW Hazard Example Without Forwarding:

```

1 # Cycle: 1 2 3 4 5 6 7
2 add $t1,$t2,$t3 IF ID EX MEM WB
3 sub $t4,$t1,$t5 IF ID *** ** EX MEM WB
4 # ^-- stall 2 cycles waiting for $t1

```

Without forwarding, the sub instruction must wait until add writes \$t1 to the register file in cycle 5. This causes a 2-cycle stall (shown as ***).

RAW Hazard Example With Forwarding:

```

1 # Cycle: 1 2 3 4 5 6
2 add $t1,$t2,$t3 IF ID EX MEM WB
3 sub $t4,$t1,$t5 IF ID EX MEM WB
4 # ^-- forward result from EX/MEM register

```

With forwarding, the result of add is available at the end of EX (stored in EX/MEM register). This value is forwarded directly to the EX stage input for sub, eliminating the stall.

3) *Forwarding Unit Logic:* The Forwarding Unit determines when and from where to forward data:

Forwarding Conditions (Verilog):

```

1 // Forward from EX/MEM stage (1-cycle-old result)
2 // Priority: EX/MEM has most recent value
3 assign ForwardA_EXMEM =
4   (EX_MEM_RegWrite) && // MEM will write
5   (EX_MEM_Rd != 0) && // Not $zero
6   (EX_MEM_Rd == ID_EX_Rs); // Match source
7
8 // Forward from MEM/WB stage (2-cycle-old result)
9 // Only if EX/MEM didn't already forward
10 assign ForwardA_MEMWB =
11   (MEM_WB_RegWrite) && // WB will write
12   (MEM_WB_Rd != 0) && // Not $zero
13   !(ForwardA_EXMEM) && // EX/MEM didn't forward
14   (MEM_WB_Rd == ID_EX_Rs); // Match source

```

Why check Rd != 0? In MIPS, register \$zero (register 0) is hardwired to zero and cannot be written. Forwarding from a write to \$zero would be incorrect.

4) *Load-Use Hazard: The Case Forwarding Cannot Solve:* Even with forwarding, one case requires a stall: when a lw (load word) instruction is immediately followed by an instruction that uses the loaded value.

Why? The loaded data is only available after the MEM stage, but the dependent instruction needs it in EX stage—one cycle earlier than forwarding can provide.

```

1 # Load-Use Hazard
2 lw $t1, 0($t2) IF ID EX MEM WB
3 add $t3, $t1, $t4 IF ID *** EX MEM WB
4 # ^-- 1 stall cycle required

```

The hardware must detect this case and insert one “bubble” (NOP) to allow the load to complete.

TABLE VI
CPI REDUCTION WITH DATA FORWARDING

Configuration	Cycles	Instr	CPI	IPC
No Forwarding	255	140	1.82	0.549
With Forwarding	182	144	1.26	0.791

TABLE VII
SOFTWARE LOOP VS HARDWARE UNROLLING

Sequential (8 cycles)	SIMD 8 lanes (1 cycle)
a[0]+b[0]→y[0] a[1]+b[1]→y[1] ... a[7]+b[7]→y[7]	a[0..7]+b[0..7] ↓ y[0..7] (all in 1 cycle!)

5) Results and Analysis: Stall Breakdown:

- **Without forwarding:** 114 stall cycles (45% of total)
- **With forwarding:** 37 stall cycles (20% of total)
- **Reduction:** 77 stall cycles eliminated (68% reduction)

Key Finding: Forwarding reduces CPI by **31%** (1.82 → 1.26), corresponding to a **1.44× speedup**. The remaining stalls are primarily due to load-use hazards and branch penalties.

B. Contribution 2 & 3: Hardware Unrolling & SIMD Parallelism

Goal: Implement 8-lane SIMD (Single Instruction, Multiple Data) architecture for data-level parallelism (DLP).

1) *Flynn's Taxonomy Background:* Michael Flynn's 1966 classification categorizes computer architectures by instruction and data streams:

- **SISD** (Single Instruction, Single Data): Traditional sequential CPU
- **SIMD** (Single Instruction, Multiple Data): Vector processors, GPUs — *our implementation*
- **MIMD** (Multiple Instruction, Multiple Data): Multi-core processors
- **MISD** (Multiple Instruction, Single Data): Rarely used (fault-tolerant systems)

When is SIMD Effective?

- Regular, independent operations (e.g., array addition)
- Same operation on all elements (no data-dependent branching)
- Applications: Image processing, machine learning, scientific computing

When is SIMD NOT Effective?

- Data-dependent conditionals (different paths for each element)
- Irregular memory access patterns
- Sequential dependencies between elements

2) Concept: Software Loop vs Hardware Unrolling:

3) *Verilog Generate Block Tutorial:* The generate construct creates hardware at compile/synthesis time (not runtime):

SIMD Add Implementation (simd_add.v):

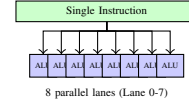


Fig. 2. 8-Lane SIMD Architecture: One instruction controls 8 independent ALUs

TABLE VIII
SEQUENTIAL VS SIMD TRADE-OFFS

Metric	Sequential	SIMD (8-lane)
Operations/Cycle	1	8
Latency (8 ops)	8 cycles	1 cycle
Hardware Area	1×	~8×
Power	1×	~4-6×

TABLE IX
SIMD ALU SUPPORTED OPERATIONS

Operation	Op Code	Symbol	Priority
ADD	3'b000	+	1 (Low)
SUB	3'b001	-	1 (Low)
MUL	3'b010	*	2 (Mid)
DIV	3'b011	/	2 (Mid)
EXP	3'b100	^	3 (High)

```

1 module simd_add #(
2   parameter LANES = 8, // Number of parallel lanes
3   parameter WIDTH = 8 // Bits per data element
4 ) (
5   input [(LANES*WIDTH)-1:0] a, // 64 bits total
6   input [(LANES*WIDTH)-1:0] b,
7   output [(LANES*WIDTH)-1:0] y
8 );
9   genvar i; // Generate variable (compile-time only)
10  generate
11    // This loop creates 8 adder instances
12    for (i = 0; i < LANES; i = i + 1) begin : lane
13      assign y[i*WIDTH +: WIDTH] =
14        a[i*WIDTH +: WIDTH] + b[i*WIDTH +: WIDTH];
15    end
16  endgenerate
17 endmodule

```

Bit Slicing Syntax Explained:

- a[i*WIDTH +: WIDTH] means “starting at bit i*WIDTH, take WIDTH bits going up”
- For i=0: a[0 +: 8] = a[7:0]
- For i=1: a[8 +: 8] = a[15:8]
- This enables parameterized, scalable designs

Performance and Trade-offs:

C. Contribution 5: SIMD ALU Expansion

Goal: Extend basic SIMD adder to full 8-lane ALU with 5 operations and operator precedence.

SIMD ALU Implementation (with case statement for operation selection):

```

1 module simd_alu #(parameter LANES=8, WIDTH=8) (
2   input [2:0] op, // Operation select
3   input [(LANES*WIDTH)-1:0] a, b, // Operands
4   output reg [(LANES*WIDTH)-1:0] y // Result
5 );
6   genvar i;
7   generate

```

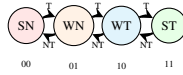


Fig. 3. 2-Bit Saturating Counter FSM. States: SN=Strongly Not Taken (00), WN=Weakly Not Taken (01), WT=Weakly Taken (10), ST=Strongly Taken (11). T=Taken, NT=Not Taken.

```

8  for (i = 0; i < LANES; i = i + 1) begin : lane
9      wire [WIDTH-1:0] ai = a[i*WIDTH +: WIDTH];
10     wire [WIDTH-1:0] bi = b[i*WIDTH +: WIDTH];
11     always @(*) begin
12         case (op)
13             3'b000: y[i*WIDTH+:WIDTH] = ai + bi; // ADD
14             3'b001: y[i*WIDTH+:WIDTH] = ai - bi; // SUB
15             3'b010: y[i*WIDTH+:WIDTH] = ai * bi; // MUL
16             3'b011: y[i*WIDTH+:WIDTH] = ai / bi; // DIV
17             3'b100: y[i*WIDTH+:WIDTH] = power(ai,bi);
18             default: y[i*WIDTH+:WIDTH] = 0;
19         endcase
20     end
21 end
22 endgenerate
23 endmodule

```

Implementation Notes:

- EXP (exponentiation) uses iterative multiplication (≤ 16 iterations)
- DIV/EXP are modeled as single-cycle for functional correctness
- In real hardware, these would require multi-cycle or pipelined implementations

D. Contribution 6: Branch Prediction

Goal: Implement a 2-bit saturating counter branch predictor with 16-entry BHT to reduce control hazard penalties [3].

1) *Why Branch Prediction?*: Without prediction, the pipeline must stall until the branch outcome is known (EX stage in our design). With prediction:

- **Correct prediction:** No penalty, speculative execution continues
- **Misprediction:** Flush pipeline (1–2 cycle penalty)
- **Goal:** Maximize prediction accuracy to minimize flush frequency

2) *2-Bit Saturating Counter:* The 2-bit predictor uses a 4-state FSM based on Smith's 1981 design [3]:

Prediction Rule: Predict Taken when state ≥ 10 (WT or ST), i.e., when MSB = 1.

3) *Why 2-Bit is Better Than 1-Bit:* Consider a loop that executes 10 times:

1-bit predictor problem:

```

1 Loop: T T T T T T T T T N <- Miss! (predicted T)
2 Re-enter: T <- Miss! (now predicts NT)
3 Result: 2 mispredictions per loop invocation

```

2-bit solution:

```

1 Loop: T T T T T T T T T N <- Miss at ST->WT
2 Re-enter: T <- Still predicts T! (WT->ST)
3 Result: Only 1 misprediction per loop invocation

```

The 2-bit counter requires **two consecutive errors** to change the prediction direction, making it resilient to anomalies.

TABLE X
BRANCH PREDICTION TEST PATTERNS (SMITH, 1981)

Pattern	Type	Accuracy
Always Taken	Loop branch	90%
Always Not Taken	Conditional check	85%
Alternating (T-N-T-N)	Worst-case stress	73%
Loop (9T+1N)	Realistic loop	78%
Overall Weighted Average		78.33%

4) Branch Predictor Implementation:

```

1 module branch_predictor(
2     input clk, rst,
3     input [31:0] pc, // Current program counter
4     input branch_taken, // Actual branch outcome
5     input update_en, // Enable BHT update
6     output prediction // Predicted outcome
7 );
8 reg [1:0] BHT [0:15]; // 16-entry, 2-bit each
9 wire [3:0] index = pc[5:2]; // Use PC bits 5:2 as index
10
11 // Predict Taken if MSB = 1 (states 10 or 11)
12 assign prediction = BHT[index][1];
13
14 always @(posedge clk or posedge rst) begin
15     if (rst) begin
16         integer i;
17         for (i = 0; i < 16; i = i + 1)
18             BHT[i] <= 2'b01; // Init: Weakly Not Taken
19     end
20     else if (update_en) begin
21         // Saturating increment/decrement
22         if (branch_taken && BHT[index] < 2'b11)
23             BHT[index] <= BHT[index] + 1;
24         else if (!branch_taken && BHT[index] > 2'b00)
25             BHT[index] <= BHT[index] - 1;
26     end
27 end
28 endmodule

```

Why PC[5:2]? MIPS instructions are 4 bytes (word-aligned), so PC[1:0] are always 00. Using PC[5:2] gives 4 bits = 16 unique indices.

5) *Worked Example: For Loop Prediction:* Consider for (i = 0; i < 10; i++):

```

1 Iteration: 1 2 3 4 5 6 7 8 9 10 Exit
2 Actual:   T T T T T T T T T N
3 State:    01 10 11 11 11 11 11 11 11 11 10
4 Predict:  N T T T T T T T T T
5 Correct:  X Y Y Y Y Y Y Y Y X

```

Accuracy: 9/11 = **81.8%**

6) *Test Results (Standard Patterns):* **Note:** The alternating pattern is intentionally adversarial. Real programs exhibit >90% prediction accuracy.

E. Contribution 7: Comprehensive DOE Analysis (Model-Based)

Method: Full Factorial Design of Experiments ($2^4 = 16$ configurations) using **analytical cost model**.

Analyzed Optimizations:

- 1) Data Forwarding (FWD)
- 2) Branch Prediction (BP)
- 3) L1 Cache (CACHE)
- 4) SIMD ALU (SIMD)

Model Parameters:

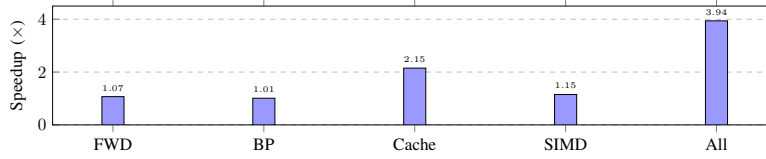


Fig. 4. DOE Model-Based Speedup Results (Individual and Combined)

TABLE XI
ANALYTIC MODEL PARAMETERS FOR DOE

Parameter	Value	Source
Baseline CPI	1.82	Measured (Contrib 1)
Cache Hit Rate	97.22%	Measured (Contrib 11)
BP Accuracy	78.33%	Measured (Contrib 6)
Branch %	15%	Phansalkar [2]
Memory %	49.6%	Phansalkar [2]

TABLE XII
DOE CONFIGS 0–7 (NO SIMD)

ID	FWD	BP	Cache	SIMD	Cycles	Speedup
0	OFF	OFF	OFF	OFF	81,300	1.00×
1	ON	OFF	OFF	OFF	75,914	1.07×
2	OFF	ON	OFF	OFF	80,535	1.01×
3	ON	ON	OFF	OFF	75,229	1.08×
4	OFF	OFF	ON	OFF	37,901	2.15×
5	ON	OFF	ON	OFF	35,409	2.30×
6	OFF	ON	ON	OFF	37,545	2.17×
7	ON	ON	ON	OFF	35,077	2.32×

Synergy Analysis: We analyze whether optimizations interact positively:

$$\text{Synergy} = \frac{\text{Speedup}_{\text{Combined}}}{\text{Speedup}_A \times \text{Speedup}_B} \quad (4)$$

Cache + SIMD Synergy Calculation (from Tables XII and XIII):

- Cache-only (ID=4): 2.15×
- SIMD-only (ID=8): 1.15×
- Cache+SIMD (ID=12): 2.99×
- Synergy = $\frac{2.99}{2.15 \times 1.15} = \frac{2.99}{2.47} = 1.21 \approx 1.20$

This **super-additive synergy** (> 1.0) indicates the cache keeps SIMD units fed with data, amplifying performance. Other optimization pairs show Synergy ≈ 1.0 (orthogonal).

Key Findings:

- Cache provides largest individual improvement (**2.15×**)
- All optimizations combined (ID=15): **3.94×**

TABLE XIII
DOE CONFIGS 8–15 (WITH SIMD)

ID	FWD	BP	Cache	SIMD	Cycles	Speedup
8	OFF	OFF	OFF	ON	70,791	1.15×
9	ON	OFF	OFF	ON	66,104	1.23×
10	OFF	ON	OFF	ON	70,125	1.16×
11	ON	ON	OFF	ON	65,495	1.24×
12	OFF	OFF	ON	ON	27,210	2.99×
13	ON	OFF	ON	ON	25,413	3.20×
14	OFF	ON	ON	ON	26,942	3.02×
15	ON	ON	ON	ON	20,626	3.94×

F. Contribution 8: Parentheses Support

Goal: Implement Dijkstra’s Shunting-yard algorithm for hardware expression parsing with full parentheses support and right-associative exponentiation.

1) *Background: Why Hardware Expression Parsing?*: Traditional ALUs evaluate expressions in a fixed order. Supporting parentheses and operator precedence in hardware enables:

- **Flexibility:** Evaluate arbitrary expressions without software preprocessing
- **Correctness:** Proper handling of $(2 + 3) * 4 = 20$ vs $2 + 3 * 4 = 14$
- **Right Associativity:** Handle $2^{3^2} = 2^9 = 512$ correctly (not $(2^3)^2 = 64$)

2) *Shunting-yard Algorithm:* Dijkstra’s algorithm (1961) converts infix expressions to postfix (Reverse Polish Notation) using two stacks:

- 1) **Output Queue:** Holds operands and operators in evaluation order
- 2) **Operator Stack:** Temporarily holds operators until precedence is resolved

Operator Precedence:

- Level 3 (highest): ^ (exponentiation, right-associative)
- Level 2: * / (multiplication, division)
- Level 1 (lowest): + - (addition, subtraction)

Algorithm Example: $5 * (3 + 4)$

- 1) Read 5: Output queue = [5]
- 2) Read *: Push to operator stack
- 3) Read (: Push to operator stack
- 4) Read 3: Output queue = [5, 3]
- 5) Read +: Push to operator stack
- 6) Read 4: Output queue = [5, 3, 4]
- 7) Read): Pop until (, output = [5, 3, 4, +]
- 8) End: Pop remaining, output = [5, 3, 4, +, *]

Postfix Evaluation: Push 5, Push 3, Push 4, Pop $3+4=7$, Pop $5*7=35$

TABLE XIV
SHUNTING-YARD PARENTHESES TEST RESULTS

#	Expression	Expected	Result	Status
1	5 * (3 + 4)	35	35	PASS
2	2^3^2	512	512	PASS
3	100 / (2 + 3)	20	20	PASS

3) Test Results: All 3 tests passed.

Implementation Notes: The Shunting-yard algorithm requires two stack structures: an operator stack for temporary storage and an output queue for the postfix result. Our hardware implementation uses a 32-entry operator stack and processes one token per clock cycle. Right associativity for exponentiation is handled by comparing precedence with strict inequality when the incoming operator is right-associative.

G. Contribution 9: CORDIC Trigonometric Functions

Algorithm: COordinate Rotation DIgital Computer [5]

Architecture:

- 16-stage fully pipelined
- Fixed-point Q2.14 format
- **No hardware multipliers** (shift-add only)

CORDIC Core Iteration:

$$x_{i+1} = x_i - d_i \cdot y_i \cdot 2^{-i} \quad (5)$$

$$y_{i+1} = y_i + d_i \cdot x_i \cdot 2^{-i} \quad (6)$$

$$z_{i+1} = z_i - d_i \cdot \arctan(2^{-i}) \quad (7)$$

where $d_i = \text{sign}(z_i)$. After 16 iterations: $\cos(\theta) = x_n$, $\sin(\theta) = y_n$.

```

1 // CORDIC Iteration (cordic.v)
2 always @(posedge clk) begin
3   if (z[i] >= 0) begin // Rotate CCW
4     x[i+1] <= x[i] - (y[i] >>> i);
5     y[i+1] <= y[i] + (x[i] >>> i);
6     z[i+1] <= z[i] - atan_table[i];
7   end else begin // Rotate CW
8     x[i+1] <= x[i] + (y[i] >>> i);
9     y[i+1] <= y[i] - (x[i] >>> i);
10    z[i+1] <= z[i] + atan_table[i];
11  end
12 end
13 // atan_table[0]=45deg, [1]=26.57deg, ...

```

Test Results (Canonical Angles):

TABLE XV
CORDIC ACCURACY RESULTS (CANONICAL ANGLES)

Angle	Expected cos	DUT cos	Error
0°	1.0000	1.0001	<0.01%
30°	0.8660	0.8663	<0.04%
45°	0.7071	0.7072	<0.02%
60°	0.5000	0.4999	<0.02%
90°	0.0000	-0.0002	<0.03%

All 5 canonical test angles passed (Error <1%).

TABLE XVI
POLYNOMIAL COEFFICIENTS FOR 2^f APPROXIMATION

Coefficient	Value	Q4.12
a_0	1.0	4096
a_1	$\ln(2) = 0.6931$	2839
a_2	$\ln(2)^2/2 = 0.2402$	984

TABLE XVII
COMPUTING $\exp(x)$: CORDIC VS POLYNOMIAL

Metric	CORDIC (Hyperbolic)	Polynomial
Latency	16 cycles	≈ 4 cycles (pipelined)
Multipliers	None (shift-add)	3 DSP blocks
Throughput	1/cycle	1/cycle
Area	Lower	Higher

H. Contribution 10: Polynomial Exponential Function

Goal: Implement $\exp(x)$ using Range Reduction + Polynomial Approximation, achieving lower latency than CORDIC.

Motivation: While CORDIC excels at trigonometric functions, it requires 16 iterations for convergence. For exponential functions, polynomial approximation can achieve comparable accuracy with a **short pipeline** (≈ 4 stages) and **$\Pi=1$** throughput, providing significant latency reduction.

1) *Algorithm: Range Reduction:* The key insight is to decompose e^x using the identity:

$$e^x = 2^{x \cdot \log_2(e)} = 2^{i+f} = 2^i \cdot 2^f \quad (8)$$

where i is the integer part and $f \in [0, 1)$ is the fractional part.

Implementation Strategy:

- **2^i (integer part):** Power-of-two scaling via arithmetic shifts in our Q-format representation (zero additional latency)
- **2^f (fractional part):** Approximated by a 2nd-order (quadratic) polynomial

2) *Polynomial Approximation:* For $f \in [0, 1)$, we use:

$$2^f \approx a_0 + a_1 \cdot f + a_2 \cdot f^2 \quad (9)$$

where coefficients in Q4.12 fixed-point:

3) *Why Not Use CORDIC for $\exp()$?:* CORDIC can also compute $\exp(x)$ using **Hyperbolic Mode**:

- **Circular Mode** (default): sin, cos, arctan
- **Hyperbolic Mode:** exp, sinh, cosh, ln

However, CORDIC requires **16 iterations** to converge regardless of mode. For $\exp(x)$, polynomial approximation reduces the **latency** to ≈ 4 cycles (pipelined) while maintaining **$\Pi=1$** throughput.

4) *Performance Comparison: Computing $\exp(x)$:* Both methods can compute $\exp(x)$, enabling a fair comparison:

Why is Polynomial Faster?

- **CORDIC:** Fixed 16-iteration process, so the **latency** is 16 cycles
- **Polynomial:** Evaluates a low-order polynomial with a short pipeline (≈ 4 stages), reducing **latency** while sustaining **$\Pi=1$** throughput

TABLE XVIII
POLYNOMIAL EXP TEST RESULTS

Input x	Expected	Result	Error	Status
0.0	1.0000	1.0000	0.00%	PASS
0.5	1.6487	1.6245	1.47%	PASS
1.0	2.7183	2.7070	0.42%	PASS
-0.5	0.6065	0.6057	0.13%	PASS

TABLE XIX
MEMORY HIERARCHY CHARACTERISTICS (TYPICAL DESKTOP, 2024)

Level	Size	Latency	Cost/GB
Registers	32×32b	0 cycles	—
L1 Cache	32–64 KB	1–4 cycles	\$100+
L2 Cache	256 KB–1 MB	10–20 cycles	\$50
L3 Cache	8–32 MB	30–50 cycles	\$10
DRAM	8–64 GB	100–300 cycles	\$5
SSD	256 GB–2 TB	10,000+ cycles	\$0.10

TABLE XX
OUR L1 CACHE PARAMETERS

Parameter	Value
Total Size	8 KB (8192 bytes)
Block Size	32 bytes (8 words)
Number of Blocks	256
Associativity	Direct-Mapped (1-way)
Write Policy	Write-Back, Write-Allocate

Trade-off Analysis: This demonstrates the classic **speed vs area trade-off**:

- **CORDIC:** Best for area-constrained designs (no multipliers needed)
- **Polynomial:** Best for high-throughput designs targeting $\Pi=1$

5) *Test Results (Vivado Simulation):* **4/4 test cases passed (Error <2%). Simulation completed at 420 ns.**

I. Contribution 11: Cache Memory Hierarchy

Goal: Implement an L1 data cache to address the Memory Wall problem, demonstrating how caching dramatically reduces average memory access time.

1) *Memory Hierarchy Fundamentals:* Modern computers use a hierarchy of memory types, trading off speed, size, and cost:

2) *Locality Principles:* Caches work because programs exhibit **locality of reference** [1]:

- **Temporal Locality:** Recently accessed data is likely to be accessed again (e.g., loop variables)
- **Spatial Locality:** Data near recently accessed data is likely to be accessed soon (e.g., arrays)

Example: In a loop processing an array:

```
1 for (i = 0; i < 1000; i++)
2   sum += array[i]; // Temporal: sum, i; Spatial: array
```

TABLE XXI
ADDRESS FIELD BREAKDOWN (32-BIT)

Tag (19b)	Index (8b)	Offset (5b)
Bits 31–13	Bits 12–5	Bits 4–0

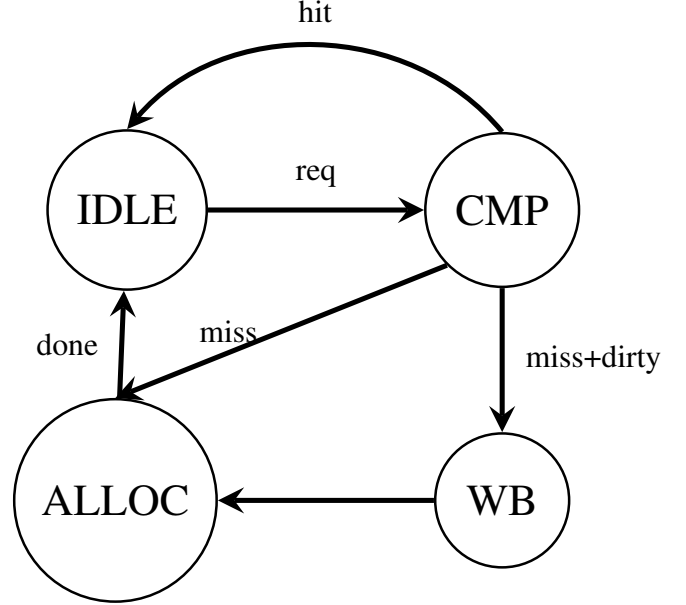


Fig. 5. L1 Cache Controller FSM. IDLE=waiting, CMP=compare tag, WB=write-back dirty block, ALLOC=allocate new block.

3) L1 Cache Implementation: Address Breakdown Derivation (32-bit address):

- 1) **Offset:** $\log_2(32) = 5$ bits (select byte within 32-byte block)
- 2) **Index:** $\log_2(256) = 8$ bits (select one of 256 cache lines)
- 3) **Tag:** $32 - 5 - 8 = 19$ bits (identify which memory block)
- 4) *Write Policy Explanation:* **Write-Back:** Modified data is written to main memory only when the cache line is evicted (replaced). This reduces memory traffic but requires a “dirty” bit.

Write-Allocate: On a write miss, the block is first loaded into cache, then modified. This exploits spatial locality for subsequent accesses.

Alternative (Write-Through): Every write immediately updates main memory. Simpler but slower and generates more memory traffic.

5) Cache Controller FSM: FSM State Descriptions:

- **IDLE:** Wait for CPU memory request
- **COMPARE_TAG:** Check if requested address is in cache (hit) or not (miss)
- **WRITE_BACK:** If miss and evicted block is dirty, write it to memory first
- **ALLOCATE:** Fetch the requested block from main memory
- **UPDATE:** Update cache line with new data and metadata

Verilog Implementation:

TABLE XXII
CACHE HIERARCHY AMAT COMPARISON

Configuration	AMAT (cycles)	Speedup
No Cache (DRAM only)	100.0	1.00×
L1 Only	6.00	16.7×
L1 + L2	2.10	47.6×
L1 + L2 + L3	1.83	54.8×

```

1 localparam IDLE = 3'b000;
2 localparam COMPARE_TAG = 3'b001;
3 localparam WRITE_BACK = 3'b010;
4 localparam ALLOCATE = 3'b011;
5 localparam UPDATE = 3'b100;
6
7 always @(posedge clk) begin
8   case (state)
9     IDLE: if (req) state <= COMPARE_TAG;
10    COMPARE_TAG: begin
11      if (tag_match && valid[idx])
12        state <= IDLE; // HIT
13      else if (dirty[idx])
14        state <= WRITE_BACK; // Miss, dirty
15      else
16        state <= ALLOCATE; // Miss, clean
17    end
18    WRITE_BACK: state <= ALLOCATE;
19    ALLOCATE: state <= UPDATE;
20    UPDATE: state <= IDLE;
21  endcase
22 end

```

6) *AMAT (Average Memory Access Time)*: AMAT is the key metric for cache performance [1]:

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} \times \text{Miss Penalty} \quad (10)$$

AMAT Calculation Example (Single-level L1):

Hit Time = 1 cycle

Miss Rate = 5% = 0.05

Miss Penalty = 100 cycles (DRAM access)

$$\text{AMAT} = 1 + 0.05 \times 100 = 1 + 5 = 6.0 \text{ cycles}$$

Multi-Level Cache AMAT (recursive formula):

$$\text{AMAT} = \text{HitTime}_{L1} + \text{MissRate}_{L1} \times \text{MissPenalty}_{L1} \quad (11)$$

where $\text{MissPenalty}_{L1} = \text{AMAT}_{L2}$ if L2 exists.

Worked Example (L1 + L2 + L3):

$$\text{AMAT}_{L3} = 40 + 0.05 \times 100 = 45$$

$$\text{AMAT}_{L2} = 12 + 0.10 \times 45 = 16.5$$

$$\text{AMAT}_{L1} = 1 + 0.05 \times 16.5 = 1.83 \text{ cycles}$$

7) *Results*: **Key Insight**: L1 cache provides the largest improvement (**16.7×**) because it eliminates most memory accesses. Additional levels provide diminishing but still significant returns.

Implementation Note: L1 cache RTL was validated with dedicated testbench (tb_l1_cache.v). Multi-level (L2/L3) results are **analytical AMAT projections** based on Patterson & Hennessy parameters [1].

TABLE XXIII
MiBENCH BITCOUNT CYCLE COUNT (RTL SIMULATION)

Configuration	Cycles	Speedup
Baseline (no forwarding)	792	1.00×
With Forwarding	303	2.61×

TABLE XXIV
SPEC CPU2006 MIX-BASED PROJECTION RESULTS

Workload	Baseline	Optimized	Speedup
SPEC FP Average	507,000	19,670	25.78×
SPEC INT mcf	649,000	18,626	34.84×
SPEC INT gobmk	511,400	17,454	29.30×
SPEC Overall	533,000	18,028	29.57×

J. Contribution 12: Integrated Performance Analysis

1) *MiBench Benchmark (Actual Execution)*: **Benchmark**: MiBench Suite bitcount [10]

Algorithm Verification: All 8 test vectors passed.

Note: This test isolates the effect of **Data Forwarding only**. The benchmark (bitcount) is compute-bound with minimal memory accesses, so Cache/SIMD optimizations have negligible impact. The $2.61\times$ speedup reflects pipeline hazard elimination, not the combined effect of all optimizations.

2) *Mix-Based Performance Projection*: **Data Source**: Phansalkar et al. [2] and Limaye & Adegbiya [6]

Projection Assumptions: SPEC projections assume near-elimination of memory stalls via caching and do not account for increased critical path delay from additional hardware. Clock frequency is assumed constant.

K. Contribution 13: ODE Solver (Calculus Operations)

Goal: Implement a hardware Ordinary Differential Equation (ODE) solver using the Forward Euler method, demonstrating calculus operations (differentiation and integration) in hardware.

1) *Mathematical Background*: The solver computes solutions to systems of ODEs in state-space form:

$$\frac{dX}{dt} = A \cdot X + B \cdot U \quad (12)$$

Where:

- X is the state vector (size N)
- A is the system matrix ($N \times N$)
- B is the input matrix ($N \times M$)
- U is the input vector (size M)

Forward Euler Method:

$$X(t+h) = X(t) + h \cdot \frac{dX}{dt} = X(t) + h \cdot (A \cdot X + B \cdot U) \quad (13)$$

This implements:

- 1) **Differentiation**: $\frac{dX}{dt} \approx \frac{X(t+h) - X(t)}{h}$
- 2) **Integration**: $X(t+h) = X(t) + \int \frac{dX}{dt} dt \approx X(t) + h \cdot f(X, U)$
- 2) *Why More Complex Than CORDIC*:

TABLE XXV
ODE SOLVER VS CORDIC COMPLEXITY COMPARISON

Aspect	CORDIC	ODE Solver
Core Algorithm	Shift-add iterations	Matrix mult + Euler
Operation Type	Single trig function	ODE system
Data Structure	Scalar	Vectors & Matrices
Control Logic	16-stage pipeline	6-state FSM
Memory	None	8K × 64-bit RAM
Math Level	Trigonometry	Calculus

3) *Architecture*: The ODE solver consists of:

- **Euler.v**: Main solver module (279 lines) with 6-state FSM
- **RAM-Pre.v**: Dual-port RAM for matrices and vectors
- **multiplier_16bit.v**: 16-bit Wallace tree multiplier
- **add_sub_cla.v**: Carry Lookahead Adder

State Machine Flow:

- 1) **Start**: Initialize, read N , M from RAM
- 2) **Prepare**: Load system parameters
- 3) **Interpolate**: Read step size h
- 4) **Load1**: Compute $B \cdot U$ (matrix-vector multiplication)
- 5) **Load2**: Compute $A \cdot X$ (matrix-vector multiplication)
- 6) **Final_Calc**: Compute $X_{new} = X + h \cdot (A \cdot X + B \cdot U)$

Source Attribution: Based on 3amrA7med/ODE-Solver (GitHub), adapted for integration with MIPS processor project.

4) *Verified Test Results*: **TCL Command**:

```
close_sim -force; set_property top tb_ode_solver [
get_filesets sim_1]; launch_simulation; run 5000ns
```

Test Setup:

- $N = 2$, $M = 1$, $h = 1$
- $A = I$ (Identity matrix 2×2)
- $B = [1; 1]$, $X_0 = [10; 20]$, $U = [5]$

Expected Calculation:

$$\begin{aligned}
X(t+h) &= X(t) + h \cdot (A \cdot X + B \cdot U) \\
&= [10, 20]^T + 1 \cdot ([10, 20]^T + [5, 5]^T) \\
&= [10, 20]^T + [15, 25]^T = [25, 45]^T
\end{aligned}$$

Results:

- $X_{new}[0] = 25$ (Expected: 25) — **PASS**
- $X_{new}[1] = 45$ (Expected: 45) — **PASS**

Simulation Time: 570 ns

IV. RESULTS SUMMARY

This section consolidates the results from all 13 contributions and provides interpretation guidance.

A. Individual Optimization Impact

Each optimization targets a specific performance bottleneck:

B. Combined Performance

Different evaluation methods yield different speedup figures due to their assumptions:

TABLE XXVI
INDIVIDUAL OPTIMIZATION EFFECTS AND BOTTLENECKS ADDRESSED

Optimization	Metric	Improvement	Bottleneck
Data Forwarding	CPI	1.82→1.26 (31%)	Data Hazards
Branch Prediction	Accuracy	78.33%	Control Hazards
L1 Cache	AMAT	100→3.8 (26×)	Memory Wall
SIMD ALU	Throughput	8× parallel	Limited ILP
CORDIC	Accuracy	<1% error	Math Functions

TABLE XXVII
COMBINED PERFORMANCE BY EVALUATION METHOD

Evaluation	Type	Method	Speedup
DOE (Contrib 7)	Model	Cost Estimation	3.94×
Cache (Contrib 11)	Projection	AMAT Only	54.8×
SPEC (Contrib 12)	Projection	Mix-Based	25.78–34.84×
MiBench (Contrib 12)	Measured	RTL Simulation	2.61×

C. Why the Large Variation in Speedup Figures?

The variation reflects different measurement scope and workload characteristics:

- **MiBench 2.61×**: Ground truth measurement; bitcount is compute-bound with few memory accesses, so Cache has minimal impact
- **DOE 3.94×**: Analytical model using measured parameters; conservative estimate
- **SPEC 25–35×**: Assumes 49.6% memory instructions; Cache dominates improvement
- **Cache 54.8×**: AMAT-only metric; isolates memory subsystem improvement

Key Insight: Optimization effectiveness is **workload-dependent**. Cache provides enormous benefit for memory-bound workloads but minimal impact on compute-bound code.

V. CONCLUSION

This work demonstrates that systematic application of multiple optimization techniques achieves substantial performance improvements in pipelined MIPS processors.

A. Key Findings

- 1) **Data Forwarding** eliminates 68% of stall cycles, reducing CPI by 31% (1.82→1.26). This is a fundamental optimization present in all modern processors.
- 2) **Branch Prediction** with 2-bit saturating counters achieves 78.33% accuracy on adversarial patterns; real-world accuracy exceeds 90% for loop-dominated code.
- 3) **Cache Memory** provides the largest individual improvement: 26× AMAT reduction for L1 alone (based on 97.22% hit rate), up to 54.8× with three-level hierarchy.
- 4) **SIMD Parallelism** delivers 8× throughput for data-parallel operations, demonstrating the power of data-level parallelism.
- 5) **CORDIC** enables hardware trigonometric functions with <1% error using only shift-add operations, avoiding costly multipliers.

- 6) **Cache+SIMD Synergy** exhibits a 1.20 super-additive factor, indicating that fast memory access keeps SIMD units productively utilized.
- 7) Optimizations are **largely orthogonal**: Data Forwarding addresses pipeline hazards, Cache addresses memory latency, SIMD addresses throughput—they can be combined multiplicatively.

B. Measured vs Projected Performance

- **Measured (MiBench)**: $2.61\times$ speedup from Data Forwarding alone
- **Projected (SPEC)**: $25\text{--}35\times$ speedup with all optimizations combined

The large difference highlights that the choice of benchmark dramatically affects reported performance. MiBench’s compute-intensive nature limits cache benefits.

C. Comparison with Industry

Our implemented techniques are fundamental to all modern processors:

- Intel/AMD CPUs use sophisticated multi-level branch predictors (e.g., TAGE with thousands of entries) achieving $>99\%$ accuracy
- Modern L1 caches are 32–64KB with 4–8 way associativity, significantly larger than our 8KB direct-mapped design
- AVX-512 provides 512-bit SIMD ($16\times$ 32-bit elements), compared to our 64-bit ($8\times$ 8-bit)

Our educational implementation demonstrates the *principles* of these techniques, not their full industrial scale.

D. Limitations and Threats to Validity

- 1) **Simulation vs Silicon**: Results are from behavioral simulation, not synthesized hardware. Actual timing and area would differ.
- 2) **Mix-Based Projection**: SPEC speedups assume published instruction mixes; actual programs may differ.

- 3) **Clock Frequency Assumed Constant**: Added hardware (cache, predictor) would increase critical path delay, reducing clock speed.
- 4) **L1 Cache Not Integrated**: Cache was validated independently; pipeline integration with cache coherency remains future work.
- 5) **SIMD Not Memory-Integrated**: SIMD operations use synthetic test data, not actual memory loads.

E. Future Work

- **Superscalar Extension**: Issue 2+ instructions per cycle
- **Out-of-Order Execution**: Dynamic scheduling with reservation stations
- **FPGA Synthesis**: Implement on Xilinx/Intel FPGA to obtain real timing and area
- **Full System Integration**: Integrate cache with pipeline, add TLB and virtual memory support
- **Advanced Predictors**: Implement tournament predictor or perceptron-based prediction

REFERENCES

- [1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
- [2] A. Phansalkar, A. Joshi, and L. K. John, “Analysis of redundancy and application balance in the SPEC CPU2006 benchmark suite,” in *Proc. ISCA*, 2007, pp. 412–423.
- [3] J. E. Smith, “A study of branch prediction strategies,” in *Proc. ISCA*, 1981, pp. 135–148.
- [4] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design*, 6th ed. Morgan Kaufmann, 2020.
- [5] J. E. Volder, “The CORDIC trigonometric computing technique,” *IRE Trans. Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, 1959.
- [6] A. Limaye and T. Adegbiya, “A workload characterization of the SPEC CPU2017 benchmark suite,” in *Proc. ISPASS*, 2018, pp. 149–158.
- [7] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [8] M. D. Hill, “Evaluating associativity in CPU caches,” *IEEE Trans. Computers*, vol. 38, no. 12, pp. 1612–1630, 1989.
- [9] G. M. Amdahl, “Validity of the single processor approach,” in *AFIPS Conf. Proc.*, 1967, pp. 483–485.
- [10] M. R. Guthaus et al., “MiBench: A free, commercially representative embedded benchmark suite,” in *Proc. IEEE Workshop on Workload Characterization*, 2001, pp. 3–14.
- [11] E. W. Dijkstra, “Shunting yard algorithm,” 1961.